

ГИБКИЕ ТЕХНОЛОГИИ РАЗРАБОТКИ ПО

Гибкая методология разработки программного обеспечения ориентирована на использование итеративного подхода, при котором программный продукт создается постепенно, небольшими шагами, включающими реализацию определенного набора требований. При этом предполагается, что требования могут изменяться. Команды, использующие гибкие методологии, формируются из универсальных разработчиков, которые выполняют различные задачи в процессе создания программного продукта.

При использовании гибких методологий минимизация рисков осуществляется путём сведения разработки к серии коротких циклов, называемых *итерациями*, продолжительностью 2–3 недели. Итерация представляет собой набор задач, запланированных на выполнение определенный период. В каждой итерации создается работоспособный вариант программной системы, в которой реализуются наиболее приоритетные (для данной итерации) требования заказчика. На каждой итерации выполняются все задачи, необходимые для создания работоспособного программного обеспечения: планирование, анализ требований, проектирование, кодирование, тестирование и документирование. Хотя отдельная итерация, как правило, недостаточна для выпуска новой версии продукта, подразумевается, что текущий программный продукт готов к выпуску в конце каждой итерации. По окончании каждой итерации команда выполняет переоценку приоритетов требований к программному продукту, возможно, вносит коррективы в разработку системы.

Принципы и значение гибкой разработки

Для методологии гибкой разработки декларированы ключевые постулаты, позволяющие командам достигать высокой производительности:

- люди и их взаимодействие;
- доставка работающего программного обеспечения;
- сотрудничество с заказчиком;
- реакция на изменение.

Люди и взаимодействие. Люди – важнейшая составная часть успеха. Отдельные члены команды и хорошие коммуникации важны для высокопроизводительных команд. Для содействия коммуникации гибкие методы предполагают частые обсуждения результатов работы и внесение изменений в решения. Обсуждения могут проводиться ежедневно длительностью несколько минут и по завершении каждой итерации с анализом результатов работ и ретроспективой. Для эффективных коммуникаций при проведении собраний участники команд должны придерживаться следующих ключевых правил поведения:

- уважать мнение каждого участника команды;
- быть правдивым при любом общении;
- прозрачность всех данных, действий и решений;
- уверенность, что каждый участник поддержит команду;
- приверженность команде и ее целям.

Для создания высокопроизводительных команд в гибких методологиях кроме эффективной команды и хороших коммуникаций необходим совершенный программный инструментарий.

Работающее программное обеспечение важнее всеобъемлющей документации. Все гибкие методологии выделяют необходимость доставки заказчику небольших фрагментов работающего программного обеспечения через заданные интервалы. Программное обеспечение, как правило, должно пройти уровень модульного тестирования, тестирования на уровне системы. При этом объем документации должен быть минимальным. В процессе проектирования команда должна поддерживать в актуальном состоянии короткий документ, содержащий обоснования решения и описание структуры.

Сотрудничество с заказчиком важнее формальных договоренностей по контракту. Чтобы проект успешно завершился, необходимо регулярное и частое общение с заказчиком. Заказчик должен регулярно участвовать в обсуждении принимаемых решений по программному обеспечению, высказывать свои пожелания и замечания. Вовлечение заказчика в процесс разработки программного обеспечения необходимо для создания качественного продукта.

Оперативное реагирование на изменения важнее следования плану. Способность реагирования на изменения во многом определяет успех программного проекта. В процессе создания программного продукта очень часто изменяются требования заказчика. Заказчики очень часто точно не знают, чего хотят, до тех пор, пока не увидят работающее программное обеспечение. Гибкие методологии ищут обратную связь от заказчиков в процессе создания программного продукта. Оперативное реагирование на изменения необходимо для создания продукта, который удовлетворяет заказчика и обеспечит ценность для бизнеса.

Постулаты гибкой разработки поддерживаются 12 принципами [6, 8]. В конкретных методологиях гибкой разработки определены процессы и правила, которые в большей или меньшей степени соответствуют этим принципам. Гибкие методологии создания программных продуктов основываются на следующих принципах [8]:

1. *Высшим приоритетом считать удовлетворение пожеланий заказчика* посредством поставки полезного программного обеспечения в сжатые сроки с последующим непрерывным обновлением. Гибкие методики подразумевают быструю поставку начальной версии и частые обновления. Целью команды является поставка работоспособной версии в течение нескольких недель с момента начала проекта. В дальнейшем программные системы с постепенно расширяющейся функциональностью должны поставляться каждые несколько недель. Заказчик может начать промышленную эксплуатацию системы, если посчитает: она достаточно функциональна. Заказчик может просто ознакомиться с текущей версией программного обеспечения, представить свой отзыв с замечаниями.
2. *Не игнорировать изменение требований*, пусть даже на поздних этапах разработки. Гибкие процессы позволяют учитывать изменения для обеспечения конкурентных преимуществ заказчика. Команды, использующие гибкие методики, стремятся сделать структуру программы качественной, с минимальным влиянием изменений на систему в целом.

3. *Поставлять новые работающие версии ПО часто, с интервалом от одной недели до двух месяцев, отдавая предпочтение меньшим срокам. При этом ставится цель поставить программу, удовлетворяющую потребностям пользователя, с минимумом сопроводительной документации.*
4. *Заказчики и разработчики должны работать совместно на протяжении всего проекта. Считается, что для успешного проекта заказчики, разработчики и все заинтересованные лица должны общаться часто и помногу для целенаправленного совершенствования программного продукта.*
5. *Проекты должны воплощать в жизнь целеустремленные люди. Создавайте команде проекта нормальные условия работы, обеспечьте необходимую поддержку и верьте, что члены команды доведут дело до конца.*
6. *Самый эффективный и продуктивный метод передачи информации команде разработчиков и обмена мнениями внутри неё – разговор лицом к лицу. В гибких проектах основной способ коммуникации – простое человеческое общение. Письменные документы создаются и обновляются постепенно по мере разработки ПО и только в случае необходимости.*
7. *Работающая программа – основной показатель прогресса в проекте. О приближении гибкого проекта к завершению судят по тому, насколько имеющаяся в данный момент программа отвечает требованиям заказчика.*
8. *Гибкие процессы способствуют долгосрочной разработке. Заказчики, разработчики и пользователи должны быть в состоянии поддерживать неизменный темп сколь угодно долго.*
9. *Непрестанное внимание к техническому совершенству и качественному проектированию повышает отдачу от гибких технологий. Члены гибкой команды стремятся создавать качественный код, регулярно проводя рефакторинг.*
10. *Простота – искусство достигать большего, делая меньше. Члены команды решают текущие задачи максимально просто и качественно. Если в будущем возникнет какая-либо проблема, то в качественный код имеется возможность внести изменения без больших затрат.*

11. *Самые лучшие архитектуры, требования и проекты выдают самоорганизующиеся команды.* В гибких командах задачи поручаются не отдельным членам, а команде в целом. Команда сама решает, как лучше всего реализовать требования заказчика. Члены команды совместно работают над всеми аспектами проекта. Каждому участнику разрешено вносить свой вклад в общее дело. Нет такого члена команды, который единолично отвечал бы за архитектуру, требования или тесты.
12. *Команда должна регулярно задумываться над тем, как стать ещё более эффективной,* а затем соответственно корректировать и подстраивать свое поведение. Гибкая команда постоянно корректирует свою организацию, правила, соглашения и взаимоотношения.

Вышеприведенным принципам в определенной степени соответствует ряд методологий разработки программного обеспечения:

- *Agile Modeling* – набор понятий, принципов и приёмов (практик), позволяющих быстро и просто выполнять моделирование и документирование в проектах разработки программного обеспечения;
- *Agile Unified Process* (AUP) упрощенная версия IBM Rational Unified Process (RUP), которая описывает простое и понятное приближение (модель) для создания программного обеспечения для бизнес-приложений;
- *OpenUP* – это итеративно-инкрементальный метод разработки программного обеспечения. Позиционируется как лёгкий и гибкий вариант RUP;
- *Agile Data Method* – группа итеративных методов разработки программного обеспечения, в которых требования и решения достигаются в рамках сотрудничества разных кросс-функциональных команд ;
- *DSDM* – методика разработки динамических систем, основанная на концепции быстрой разработки приложений (Rapid Application Development, RAD). Представляет собой итеративный и инкрементный подход, который придаёт особое значение продолжительному участию в процессе пользователя/потребителя;

- *Extreme programming (XP)* – экстремальное программирование [8];
- *Adaptive software development (ADD)* – адаптивная разработка программ [9];
- *Feature driven development (FDD)* – разработка ориентированная на постепенное добавление функциональности;
- *Getting Real* – итеративный подход без функциональных спецификаций, использующийся для веб-приложений;
- *MSF fog Agile Software Development* – гибкая методология разработки ПО компании Microsoft;
- *Scrum* устанавливает правила управления процессом разработки и позволяет использовать уже существующие практики кодирования, корректируя требования или внося тактические изменения.

Следует отметить, что в чистом виде методологии гибкого программирования редко используются командами разработчиков. Как правило, успешные команды применяют полезные приемы и свойства нескольких процессов, подстраивая их под конкретное представление команды о гибкости процесса разработки.

Методология гибкой разработки SCRUM

Методология Scrum представляет собой итеративный процесс разработки программного обеспечения. При такой разработке для программного продукта создается много последовательных выпусков, в которых постепенно добавляется требуемая функциональность. Итеративный подход позволяет по завершении текущей итерации продемонстрировать заказчику работоспособный программный продукт, возможно с ограниченной функциональностью, получить отзыв, замечания и дополнительные требования, которые будут учтены в следующих итерациях. Основными артефактами в методологии Scrum являются рабочие элементы, отчеты, книги и панели мониторинга.

Рабочие элементы

Рабочие элементы используются для отслеживания, наблюдения за состоянием хода разработки ПО и создания отчетов. *Рабочий элемент* – это запись, которая создается в Visual Studio Team Foundation Server для задания определения, назначения,

приоритета и состояния элемента работы. Для шаблона Microsoft Visual Studio Scrum 2.2 определяет следующие типы рабочих элементов:

- Bug – ошибка;
- Task – задача;
- Product BackLog Item – невыполненная работа по продукту;
- Epic – возможности;
- Feature – функция;
- Impediment – препятствие;
- Test Case – тестовый случай.

На рис. 1 приведено меню Рабочие элементы.

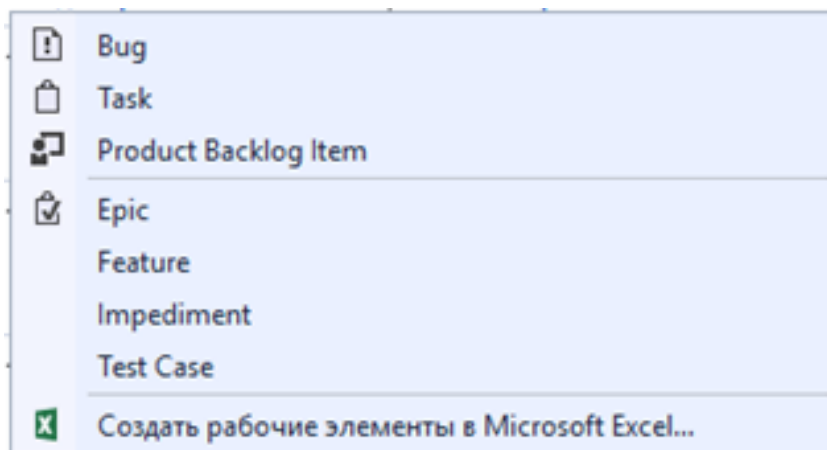


Рисунок 1 – Меню Рабочие элементы

В методологии Scrum пользовательские требования, которые определяют функциональность продукта, задаются *элементами задела работы продукта* (Product Backlog Item – PBI). Элементы задела работы продукта, которые будем называть «*элементы работы – ЭР*», представляют собой краткое описание функций продукта и оформляются в произвольной форме в виде кратких заметок. Вначале задаются наиболее важные и понятные всем пользовательские требования – ЭР. Элементы работы могут детализироваться в виде задач. В процессе создания программного продукта ЭР могут уточняться, добавляться или удаляться из списка требований.

Цикл выпуска продукта в Scrum состоит из ряда итераций, которые называются *спринтами*. Спринт имеет фиксированную

длительность, как правило, 1–4 недели. Элементы работы, включенные в очередной спринт, не подлежат изменению до его окончания. Хотя спринт завершается подготовкой работоспособного программного продукта, его текущей функциональности может быть недостаточно для оформления выпуска, имеющего ценность для заказчика. Поэтому выпуск работоспособного программного продукта, который заказчик может использовать, включает, как правило, несколько спринтов.

Организация команды

Организация команды в методологии Scrum определяет три роли [6]:

- владелец продукта (Product owner);
- руководитель (ScrumMaster);
- члены команды (Team members).

Владелец продукта отвечает за всё, что связано с потребительскими качествами программного продукта. Он определяет пользовательские требования – ЭР, анализирует их реализацию, обладает правом изменения требований, контролирует качество продукта. Он может быть представителем заказчика в команде или членом команды разработчиков, который представляет интересы заказчика. Владелец продукта выполняет следующие основные задачи:

- определение и приоритезацию требований/функций, то есть элементов работ и задач;
- планирование спринтов и выпусков;
- тестирование требований/функций.

Руководитель отвечает за состояние и координацию проекта, продуктивность команды и устранение препятствий, мешающих проекту. В обязанности руководителя входит:

- проведение ежедневных Scrum-собраний;
- привлечение сотрудников вне команды;
- стимулирование эффективного общения членов команды;
- определение размера команды.

Члены команды отвечают за разработку программного продукта высокого качества. Они должны обладать навыками в проектировании и архитектуре программного продукта, бизнес-

анализе, программировании, тестировании, настройке баз данных и проектировании пользовательского интерфейса. Члены команды участвуют в планировании спринтов. Команда может включать опытных разработчиков и новичков, которые в процессе работы должны совершенствоваться при обмене знаниями с другими членами команды. Члены команды отвечают за следующие задачи в проекте:

- обязательное выполнение элементов работ, включенных в текущий спринт;
- акцент на взаимосвязанных задачах спринта;
- совершенствование команды.

Жизненный цикл проекта ПО

Инструментальная и методическая поддержка гибкого подхода к созданию программных продуктов Scrum, реализованная в Visual Studio, позволяет управлять жизненным циклом проекта ПО (рис. 2).

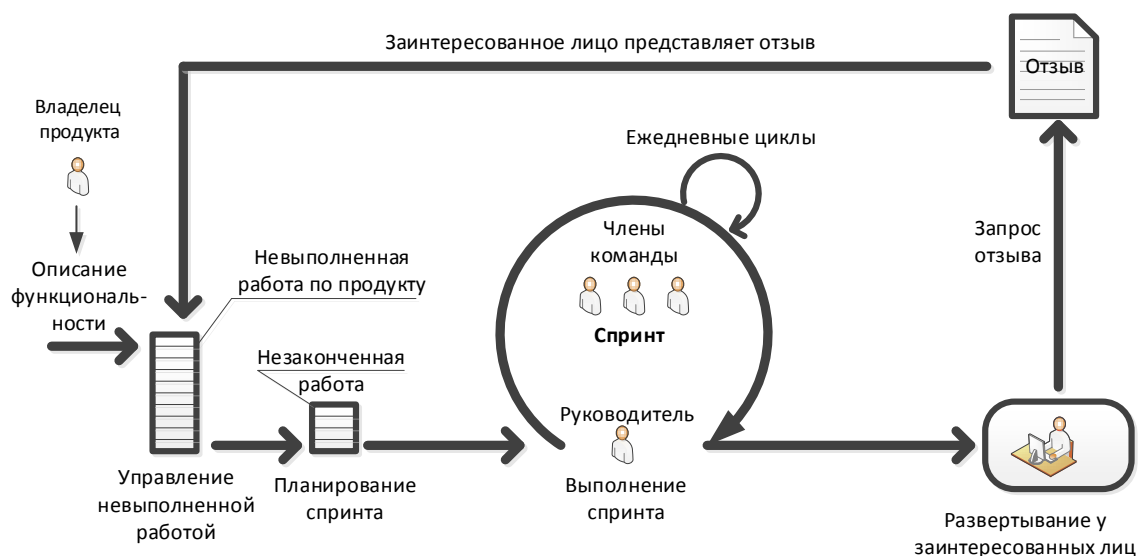


Рисунок 2 – Жизненный цикл проекта ПО

В начале проектирования владелец продукта и заказчик формируют концепцию программного продукта, которая показывает, для кого предназначен продукт, какие преимущества получают пользователи и какие существуют конкуренты. Концепция продукта связывается с областью проекта и ограничениями. Об-

ласть проекта определяет масштаб работ, а ограничения – условия, которыми будут руководствоваться для первых спринтов и выпусков. Далее владелец продукта создает список всех потенциальных функций продукта – «Невыполненная работа по продукту» (Product Backlog). *Невыполненная работа по продукту*, которую в дальнейшем будем называть *невыполненная работа – НВР*, содержит список *элементов работы* – пользовательских описаний функциональности.

Управление *невыполненной работой* по проекту сводится к поддержанию *элементов работ* в актуальном состоянии. Отдельные элементы работ списка НВР могут добавляться или удаляться в процессе создания ПО. Это является результатом того, что команда получает дополнительную информацию о новых требованиях заказчика к проектируемому программному продукту, а заказчик выясняет, как реализуются его ожидания.

Для элементов работ владелец продукта совместно с командой проекта расставляет приоритеты. При планировании спринта в него включают наиболее значимые, с точки зрения владельца продукта, пользовательские требования – ЭР, которые характеризуются наибольшей потребительской ценностью. Выбранные элементы работ перемещают в список «*Незаконченная работа*» (Sprint Backlog). Список *Незаконченная работа* (НЗР) отражает состав работ планируемого спринта. Список НЗР является результатом процесса планирования спринта.

Координацией работ в спринте занимается руководитель спринта (ScrumMaster). Он организывает процесс приоритезации задач спринта, распределения задач между членами команды. Руководитель спринта проводит собрание по планированию работ, ежедневные собрания для краткого обсуждения результатов работы и проблем, обзорные собрания в конце спринта и выпуска.

Ежедневные Scrum-собрания имеют продолжительность 15–30 минут. Целью таких собраний является выявление проблем, которые тормозят процесс разработки, и определить действия по их нейтрализации. Для простых проблем принимается решение по их устранению, а сложные проблемы откладываются на последующие спринты. В ходе ежедневного Scrum-собрания руководитель задает темп спринта, акцентирует команду на

наиболее важных элементах списка невыполненных работ. Каждый член команды сообщает, что было сделано вчера, что будет делать сегодня и какие имеются препятствия в работе. Если на ежедневном Scrum-собрании возникают вопросы, для решения которых необходимы специалисты, которых в команде нет, тогда руководитель берет на себя анализ и возможные пути разрешения данного вопроса.

Результатом спринта является работоспособное ПО, возможно, обладающее только частью необходимых функций программного продукта. Выпуск спринта может быть развернут у заинтересованных лиц для предварительного анализа соответствия ожиданиям заказчика. Результатом отзыва является формирование «Отзыв» (FeedBack), что может привести к изменению содержания списка *Элемент задела работы продукта*. После выполнения всех работ по программному продукту, то есть обнуления списка требований в списке *Элемент задела работы продукта* подготавливается финальный выпуск программного продукта.

Управление невыполненной работой

Список *Невыполненная работа по продукту* является одним из ключевых артефактов в методологии Scrum. Успех Scrum-команды во многом определяется качественным содержанием данного списка. Список НВР обычно включает пользовательские описания функциональности - *элементы работы*, а также может включать нефункциональные требования. Для создания списка НВР в TFS могут применяться различные клиентские сервисы (рис 5.3):

- командный обозреватель Visual Studio;
- веб доступ через Team Web Access;
- Microsoft Office Excel;
- Microsoft Office Project.

Владелец продукта на основе требований и пожеланий клиентов формирует список функций продукта в виде элементов задела работы продукта, которые помещает в список *Невыполненная работа по продукту*. При создании нового элемента невыполненной работы для него устанавливается состояние «Новый» (рис. 5.4).

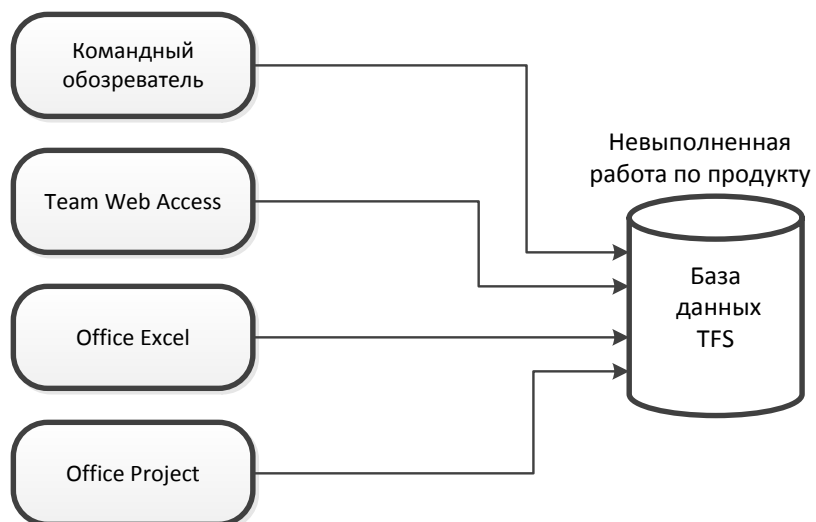


Рисунок 3 – Клиентские сервисы TFS

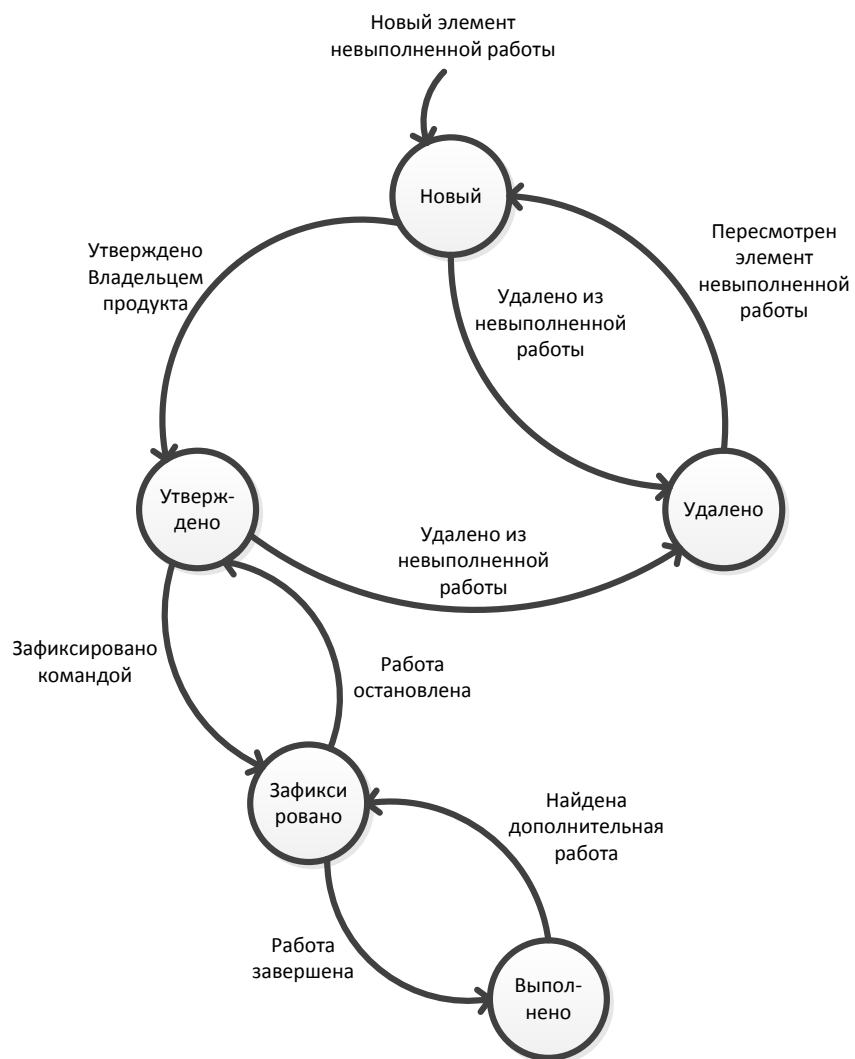


Рисунок 4 – Рабочий процесс элемента невыполненной работы

После установки элементу работы приоритета его состояние изменяют на «Утверждено». На собрании по планированию спринта команда просматривает наиболее приоритетные элементы работы и выбирает те, которые будут выполняться в текущем спринте. Для элементов невыполненной работы, которые попали в текущий спринт, устанавливается состояние «Зафиксировано». Это означает тот факт, что рабочие элементы спринта не подлежат изменению до конца спринта. При завершении работы по элементу его состояние устанавливается «Выполнено». Если для элемента работы, находящегося в состоянии «Выполнено», выявляется дополнительная работа, то этот элемент может быть переведен в состояние «Зафиксировано». Для элемента работы, находящегося в состоянии «Зафиксировано», при возникновении проблем, препятствующих его завершению в спринте, может быть работа приостановлена и установлено состояние «Утверждено». Элемент невыполненной работы может быть удален из списка *Невыполненная работа по продукту* по решению Владельца продукта. Это может произойти как из состояния «Новый», так и состояния «Утверждено», что соответствует установке для элемента работы состояния «Удалено». В результате пересмотра элемента работы, имеющего состояние «Удалено», для него вновь возможен перевод в состояние «Новый».

Список *Невыполненная работа по продукту* является главным документом для Scrum-команды. На основе данного списка команда создает другие рабочие элементы, составляющие спринты и выпуски. Для *Элементов невыполненной работы* команда создает задачи и тестовые случаи. Задачи детализируют элемент работы и определяют конкретную реализацию требований пользователя. Тестовые случаи необходимы для проверки соответствия функциональности кода требованиям пользователя. Если тестовый случай не проходит, то создается рабочий элемент «Ошибка». При блокировании задачи из-за невозможности её выполнения в текущем спринте создают рабочий элемент «Препятствие». Scrum-команда может создавать вспомогательные рабочие элементы (ошибки и препятствия) в отношении элементов, на которые они влияют (задачи и тестовые случаи), и связывать эти элементы (рис. 5).



Рисунок 5 – Связь между рабочими элементами

Для отслеживания хода выполнения проекта можно создавать отчеты, отражающие наиболее важные данные для текущего проекта. На рис.6 приведен график выполнения работ по проекту. На рис. 7 приведен график скорости работ команды по спринту. На рис. 8 приведена панель мониторинга работ по спринту.

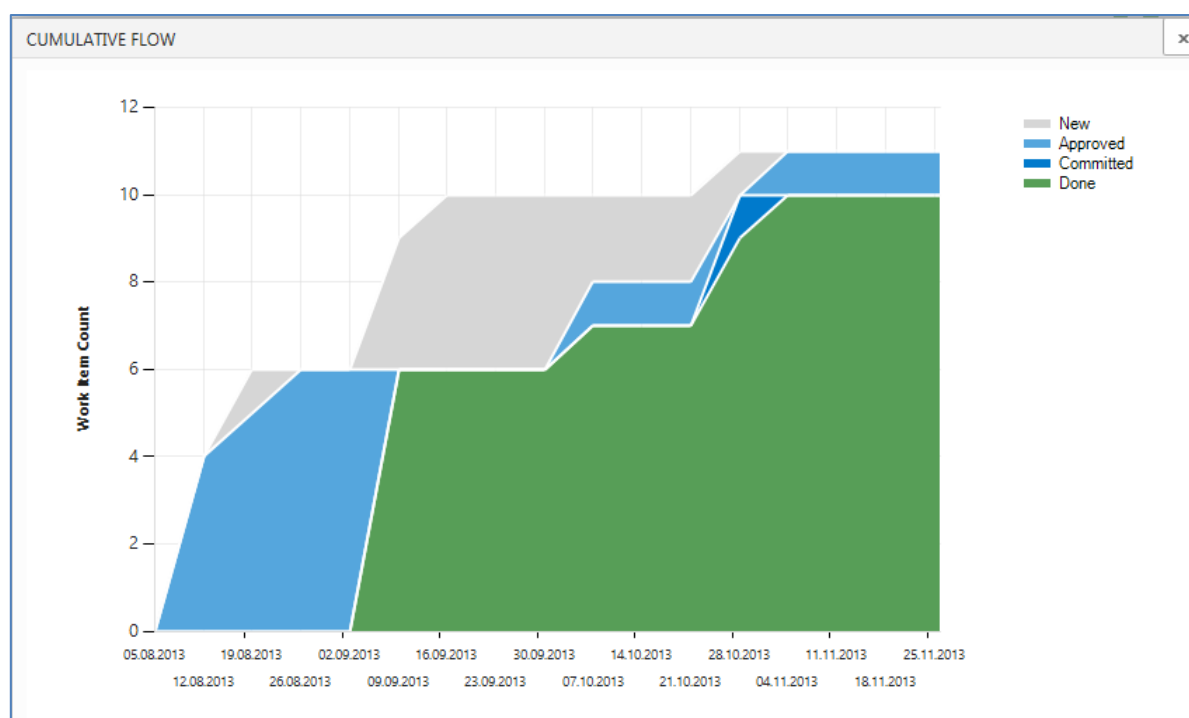


Рисунок 6 – Выполнение работ по проекту

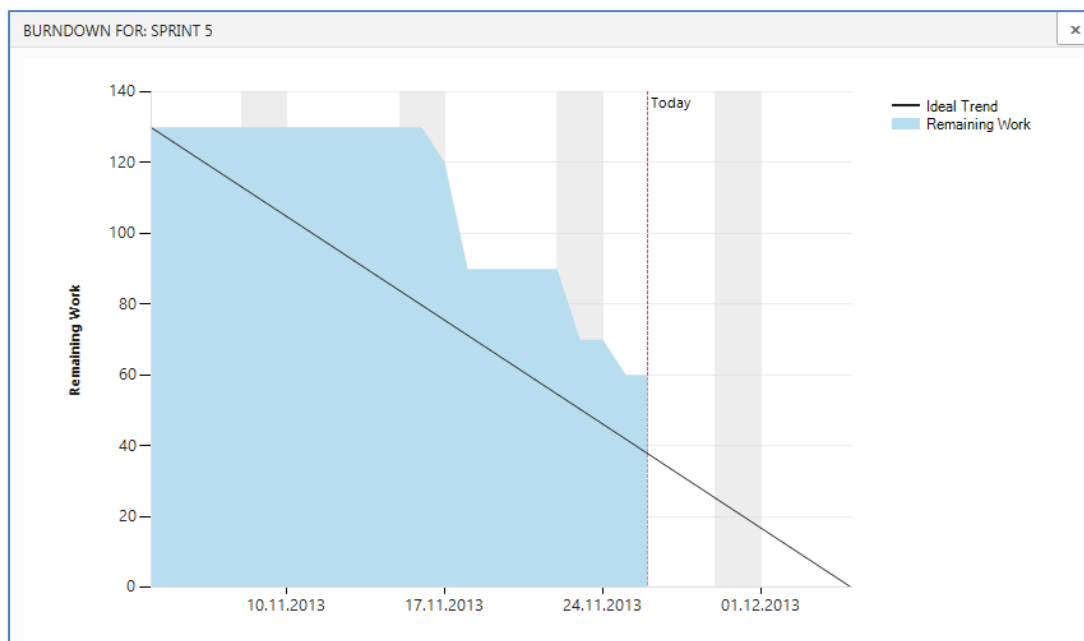


Рисунок 7 – Скорость выполнения работ по спринту

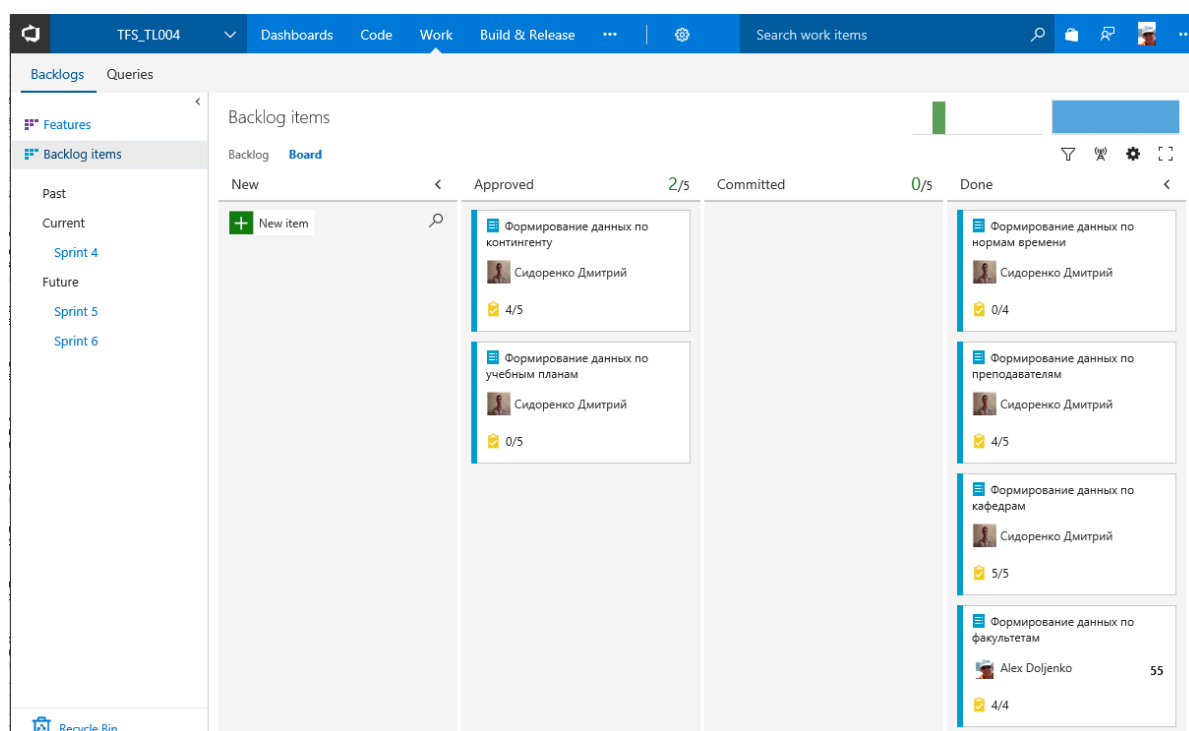


Рисунок 5 – Панель мониторинга работ по спринту

В процессе создания ПО можно пользоваться стандартными отчетами или создавать собственные отчеты. Отчеты можно создавать, настраивать и просматривать с помощью Excel, Project или служб Reporting Services SQL Server.

Методология Scrum имеет следующие положительные стороны:

- пользователи начинают видеть систему спустя всего несколько недель и могут выявлять проблемы на ранних стадиях разработки программного продукта;
- интеграция технических компонентов происходит в ходе каждого спринта, и поэтому проблемы проекта (если они возникают) выявляются практически сразу;
- в каждом спринте команда фокусируется на контроле качества;
- гибкая работа с изменениями в проекте на уровне спринта.

Ключевые термины

Гибкая методология разработки программного обеспечения – методология, ориентированная на использование итеративного подхода, при котором программный продукт создается постепенно, небольшими шагами, включающими реализацию определенного набора требований.

Итерация – набор задач, запланированных на выполнение определенный период.

Методология Scrum – гибкая методология разработки программного обеспечения, которая представляет собой итеративный процесс.

Рабочий элемент – запись, которая создается в Visual Studio Team Foundation Server для задания определения, назначения, приоритета и состояния элемента работы.

Элементы задела работы продукта – список пользовательских требований, которые определяют функциональность продукта, в методологии Scrum.

Элемент работы – краткое описание функций продукта, в методологии Scrum.

Спринт – набор задач, запланированных на выполнение определенный период времени, в методологии Scrum.

Владелец продукта – член Scrum-команды, который отвечает за всё, что связано с потребительскими качествами программного продукта.

Руководитель – член Scrum-команды, который отвечает за состояние и координацию проекта, продуктивность команды и устранение препятствий, мешающих проекту.

Член команды – член Scrum-команды, который наравне с другими членами команды отвечает за разработку программного продукта высокого качества.

Невыполненная работа по продукту – список элементов работы (пользовательских описаний функциональности), которые необходимо выполнить при создании программного продукта.

Незаконченная работа – список элементов работы планируемого спринта.

Краткие итоги

Гибкая методология разработки программного обеспечения ориентирована на использование итеративного подхода, при котором программный продукт создается постепенно. Программный продукт создается за несколько итераций, включающих реализацию определенного набора требований. Итерации имеют длительность 2–3 недели. Результатом итерации является промежуточный вариант работоспособного программного обеспечения. Для методологии гибкой разработки декларированы ключевые постулаты и принципы. Принципам гибкой разработки ПО, в определенной степени, соответствуют ряд методологий. Методология Scrum представляет собой итеративный процесс разработки программного обеспечения. Рабочие элементы используются для отслеживания, наблюдения за состоянием хода разработки ПО и создания отчетов. Организация команды в методологии Scrum определяет роли владельца продукта, руководителя и членов команды. Жизненный цикл проекта ПО включает формирование и управление невыполненной работой, планирование и выполнение спринта, развертывание ПО у заинтересованных лиц, получение отзывов для улучшения программного продукта. При управлении рабочим процессом элементов невыполненной работы по продукту постоянно отслеживаются связи и изменяется состояние этих элементов.